DEEP LEARNING
UNIT – V | PART B: LONG ANSWERS

B.Tech V Semester – AI & ML | MIC23 Regulation
Subject Code: 23AM5T01

10-MARK EXAM PREPARATION

All 9 Part B questions from the supplied Master Question Bank. Part A and Part C are excluded.

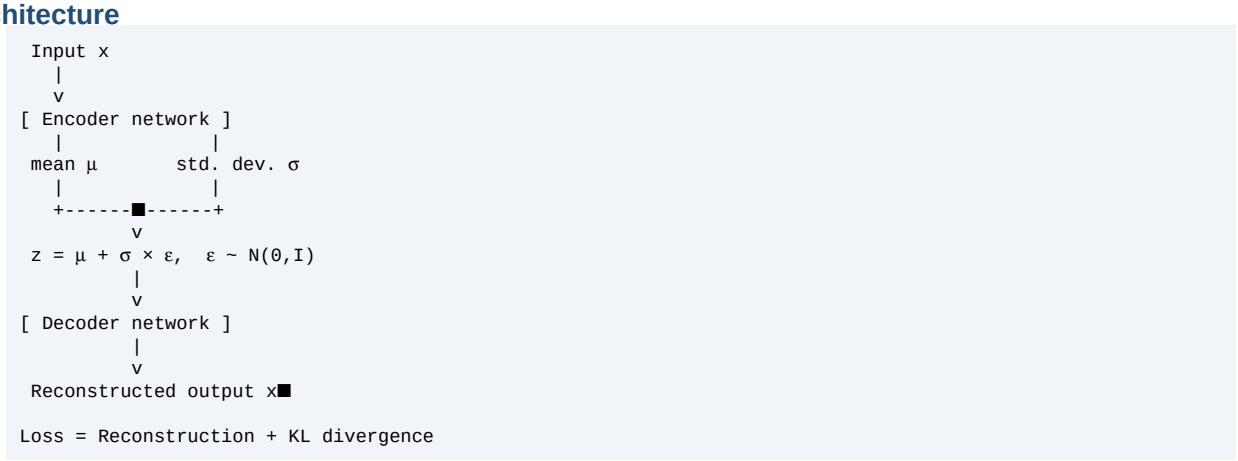
Introduction • Neat diagrams • Working • Equations • Main points • Conclusion

Q1. Explain the architecture and working of a Variational Autoencoder (VAE) with a neat diagram. Explain the Encoder, Latent Space (μ, σ), Reparameterization Trick, Decoder, and the Loss Function (Reconstruction Loss + KL Divergence).

1. Introduction

A **Variational Autoencoder (VAE)** is a generative neural network that learns a probability distribution over compact latent representations. Unlike a basic autoencoder that produces one fixed code, a VAE learns the mean and variance of a latent distribution and samples from it. It can reconstruct inputs and generate new data.

2. Architecture



The encoder and decoder are neural networks. The encoder maps input x to parameters of a latent probability distribution; the decoder maps a sampled latent vector back to the data space.

3. Encoder and latent space

The encoder produces a mean vector μ(x) and standard deviation σ(x), often by predicting log σ² for numerical stability. The approximate posterior is **qϕ(z|x)=N(z; μ(x), diag(σ²(x)))**. The latent representation is therefore a distribution, not a single deterministic point. The prior is commonly p(z)=N(0,I), encouraging a smooth and sampleable latent space.

4. Reparameterization trick

Direct random sampling is not differentiable with respect to μ and σ. VAE uses **ε ~ N(0,I)** and **z = μ + σ ■ ε**. Randomness is isolated in ε, while z remains differentiable with respect to encoder outputs. This allows gradients to pass through the sampling step during backpropagation.

5. Decoder

The decoder receives z and produces a reconstruction x■ or parameters of an output distribution. For image data it may predict pixel probabilities or pixel values. For generation, a new z is sampled from N(0,I) and passed through the decoder to create a new sample.

6. Loss function

The VAE objective is **L = Lreconstruction + LKL**. Reconstruction loss measures the difference between input x and reconstruction x■; it may use binary cross-entropy or mean squared error depending on the data. KL divergence regularizes the approximate posterior toward the prior. For a diagonal Gaussian:
DKL = ½ Σj (μj² + σj² − log σj² − 1).
The reconstruction term preserves input information, while KL divergence makes the latent space smooth. A β-VAE multiplies the KL term by β.

7. Training steps

1. Input x is passed to the encoder. 2. Encoder computes μ and σ. 3. Sample ε and calculate z=μ+σ■ε. 4. Decoder produces x■. 5. Calculate reconstruction loss and KL divergence. 6. Add losses and update parameters using backpropagation. 7. For generation, sample z from the prior and decode it.

8. Advantages and limitations

Advantages: continuous latent space, probabilistic generation, useful interpolation and end-to-end training. **Limitations:** outputs can be blurrier than GAN outputs; loss balance matters; posterior collapse may occur if the decoder ignores z.

9. Conclusion

A VAE combines an encoder, probabilistic latent space, reparameterization trick and decoder. Its reconstruction-plus-KL objective enables representation learning and generation of new samples.

Q2. Explain the complete architecture of the Transformer model with a neat schematic diagram. Describe the functions of its major components: Multi-Head Self-Attention, Positional Encoding, Scaled Dot-Product Attention, and Feed-Forward sublayers.

1. Introduction

A **Transformer** processes sequences using attention rather than recurrence. It models relationships between tokens and supports parallel processing during training. The original architecture contains stacked encoder and decoder layers.

2. Overall architecture

The encoder creates contextual representations. The decoder uses masked self-attention and encoder outputs to generate target tokens.

3. Embeddings and positional encoding

Tokens are converted into dense embedding vectors. Attention alone does not represent token order, so positional information is added. Original sinusoidal encodings are:
PE(pos,2i)=sin(pos/10000^(2i/d))
PE(pos,2i+1)=cos(pos/10000^(2i/d)).
Here pos is position, i is dimension index and d is embedding dimension.

4. Query, Key and Value

Learned projections create **Q=XWQ, K=XWK, V=XWV**. Query represents what a token is looking for; Key represents what a token offers for matching; Value contains information to be combined. QK■ produces similarity scores, which determine the weights applied to V.

5. Scaled Dot-Product Attention

The formula is **Attention(Q,K,V)=softmax(QK■/√dk)V**. Dot products measure query-key similarity. Division by √dk controls score magnitude; softmax converts scores into weights; the weighted sum of values gives the attention output.

6. Multi-Head Self-Attention

Several attention heads operate in parallel using different learned projections. Each head may learn different relationships, such as syntactic or long-distance dependencies.
head_i=Attention(QWQi, KWKi, VWVi)
MultiHead=Concat(head■,…,head_h)WO.
In self-attention, Q, K and V come from the same sequence.

7. Feed-Forward sublayer

The position-wise feed-forward network is typically:
FFN(x)=ReLU(xW■+b■)W■+b■.
It applies two linear transformations with a nonlinear activation to each token representation. Residual connections add the sublayer input to its output, while Layer Normalization stabilizes training. Dropout may also be applied.

8. Encoder and decoder functions

The encoder layer applies multi-head self-attention followed by a feed-forward network, with residual connections and normalization. The decoder first applies masked self-attention so future target tokens are hidden, then cross-attention to encoder keys and values, and finally a feed-forward network. A linear layer and softmax produce token probabilities.

9. Advantages and conclusion

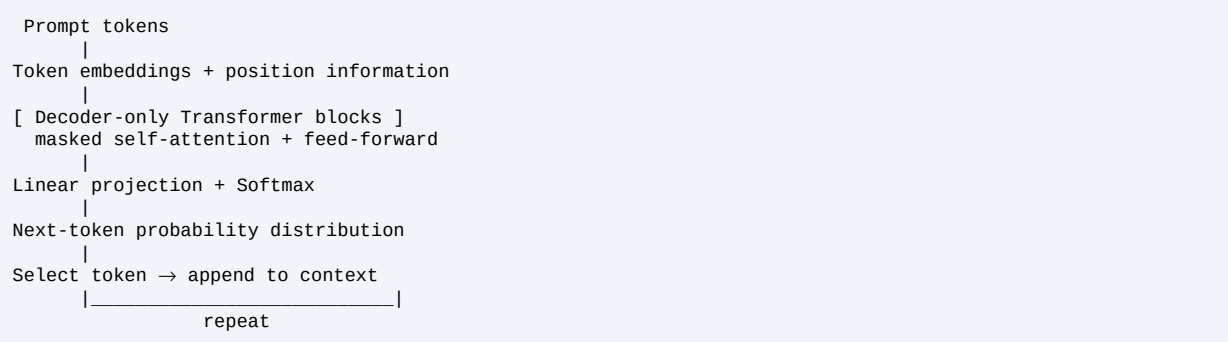
Transformers capture long-range relationships, train in parallel across sequence positions and scale effectively. They are used in translation, summarization, question answering and text generation. Attention can be computationally expensive for long sequences. The Transformer combines positional encoding, attention and feed-forward layers into a powerful sequence model.

Q3. Explain how GPT generates text using Next-Token Prediction and Autoregressive decoding. Discuss the training methodology (Pre-training + Fine-tuning / RLHF), advantages, and fundamental limitations (hallucinations, context window).

1. Introduction

GPT (Generative Pre-trained Transformer) is a decoder-only Transformer language model. It learns to predict the next token from previous tokens. Repeating this operation produces a sequence of text.

2. Architecture and generation flow



3. Next-token prediction

For tokens x■,…,x■, GPT estimates **P(x■■■ | x■,…,x■)**. During training, it learns to assign high probability to the actual next token. A common objective is negative log-likelihood:
L=−Σ**t log P(x■■■|x■,…,x■)**.
Causal masking prevents a position from using future tokens.

4. Autoregressive decoding

At inference, the model calculates next-token probabilities, selects a token, appends it to the context and predicts again. Greedy decoding selects the most probable token; beam search tracks multiple candidates; sampling methods such as temperature, top-k and top-p control diversity. Generation stops at an end token or length limit.

5. Training methodology

Pre-training: train on large text collections using next-token prediction to learn language patterns and general representations.
Supervised fine-tuning: train on curated prompt-response examples to improve instruction following.
RLHF / preference alignment: human preferences are used to train a reward model or preference objective; reinforcement learning methods such as PPO have been used to optimize responses. The exact alignment method can vary by model.

6. Advantages

GPT can draft text, answer questions, summarize, translate, explain concepts and assist with coding. One pretrained model can perform many tasks through prompts without training a separate model for each task.

7. Limitations

Hallucinations: GPT predicts plausible continuations; it does not guarantee truth and can produce confident but incorrect claims. Important information must be verified.
Context window: only a limited number of tokens can be processed in one context. Very long documents or conversations may exceed this limit.
Other limitations include bias, sensitivity to prompts, compute cost and difficulty with some exact reasoning tasks.

8. Conclusion

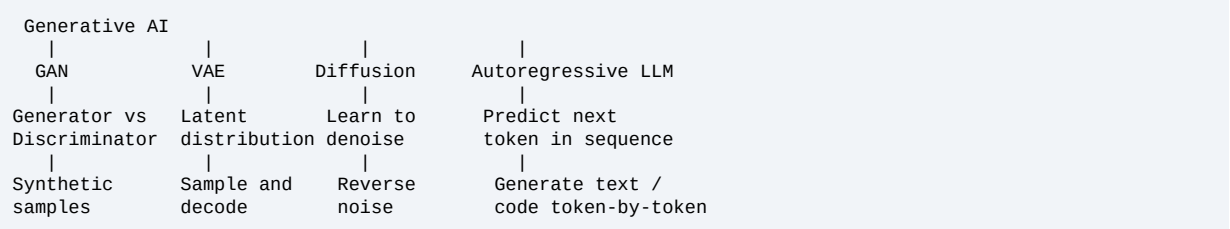
GPT generates text through repeated next-token prediction. Pre-training, fine-tuning and preference alignment improve its usefulness, but hallucinations and context limits require verification.

Q4. Define Generative AI and explain its core working principles, major model paradigms (GANs, VAEs, Diffusion Models, Autoregressive LLMs), and diverse applications in code, vision, and healthcare.

1. Definition and principles

Generative AI learns patterns or probability distributions from data and creates new content such as text, images, audio, video or code. General process: collect and preprocess data; select a model; train it to learn patterns; provide a prompt, condition or random input; generate content; evaluate quality and safety.

2. Model paradigms overview



3. GANs

A **Generative Adversarial Network** has a Generator G and Discriminator D. G maps random noise z to a fake sample G(z). D distinguishes real data from generated data. G tries to fool D while D learns to detect fake samples.
minG maxD [Ex log D(x) + Ez log(1−D(G(z)))].
GANs can generate sharp images, but training may be unstable and mode collapse can occur.

4. VAEs

A VAE encoder estimates a latent distribution q(z|x), samples z using the reparameterization trick and a decoder reconstructs or generates data. Its loss combines reconstruction error and KL divergence from a prior. VAEs offer smooth latent representations, though some generated images may look blurry.

5. Diffusion models

Diffusion models learn to reverse a gradual noising process. Training adds noise to real data over time and trains a network to predict/remove that noise. Generation starts from random noise and repeatedly denoises it.
DIAGRAM: x■ → x■ → x■ → … → xT (noise) reverse learned denoising xT → … → x■ → x■ → x■ (sample)
They are powerful for image generation and editing, but iterative sampling can be costly.

6. Autoregressive LLMs

Autoregressive models such as GPT factorize sequence probability as **P(x■,…,x■)=Π**t **P(x■|x■,…,x■■■)**. They generate one token at a time and can produce text, code and structured outputs.

7. Applications

Code: code completion, explanation, debugging support, test generation and documentation. Generated code must be reviewed and tested.
Vision: text-to-image generation, image editing, inpainting, super-resolution, synthetic data and captioning.
Healthcare: documentation assistance, patient-friendly explanations, research support, synthetic data and molecule-design exploration. Clinical use requires validation, privacy safeguards and professional oversight.

8. Benefits, challenges and conclusion

Benefits include faster creative work, personalization and synthetic data. Challenges include hallucination, bias, privacy, copyright, misuse and compute cost. GANs, VAEs, diffusion models and LLMs use different mechanisms to generate content; responsible deployment requires evaluation and human oversight.

Q5. Explain the working of an AI Agent through Goal Setting, Planning, Tool Use (APIs/Browsers), Observation, and Feedback Loops with a concrete real-world example.

1. Introduction

An **AI Agent** perceives a task or environment, chooses actions and works toward a goal. Unlike a one-shot response system, it can plan multiple steps, use tools, observe results and adapt its actions.

2. Agent workflow

```
 User goal
    |
Goal setting
    |
Planning / task breakdown
    |
Tool selection and execution
    |---- APIs / Browser / Database
    |
Observation of result
    |
Feedback and goal check
    |          |
Re-plan      Goal achieved
    |          |
    +---- loop  Final report
```

3. Goal setting and planning

The agent interprets the objective, constraints and expected output. It breaks a broad task into smaller sub-goals, identifies dependencies, chooses an action order and decides how to handle failure or missing information.

4. Tool use

APIs retrieve or update structured information; browsers access websites; databases provide records; code tools perform calculations. The agent selects a tool, supplies inputs and interprets its result. External or consequential actions must follow permission and confirmation rules.

5. Observation and feedback

After an action, the agent checks whether it succeeded, failed or returned incomplete data. It compares the observation with the goal and decides to continue, retry, use another tool or ask the user for clarification. This closed feedback loop makes the workflow adaptive.

6. Real-world example: trip planning

Goal: plan a trip within a budget.

1. Goal setting: identify destination, date, travellers and budget.
2. Planning: split into transport, accommodation and itinerary research.
3. Tool use: query travel APIs or websites for options.
4. Observation: inspect prices, timings and availability.
5. Feedback: if an option is unavailable or over budget, search for an alternative.
6. Completion: present the plan and obtain explicit approval before a paid booking.

7. Advantages, challenges and conclusion

Agents automate multi-step work and adapt to changing results. Risks include incorrect plans, tool failures, privacy issues and unintended actions. Use access controls, validation, monitoring and human approval for consequential steps. An AI Agent combines goal setting, planning, tool use, observation and feedback to complete real-world tasks.

Q6. Discuss the architectural evolution of GPT models from GPT-1 to GPT-4+ (scaling parameters, context lengths, decoder-only configuration, multimodality).

1. Introduction

GPT means **Generative Pre-trained Transformer**. GPT models use a decoder-only Transformer and autoregressive next-token prediction. Their evolution includes changes in scale, training, instruction following, context handling and modalities.

2. Decoder-only architecture

```
 Input tokens
      |
Token embedding + position
      |
Causal masked self-attention
      |
Feed-forward network
      |
Residual connections + normalization
      |
Repeated decoder blocks
      |
Language-model head + Softmax
      |
Next-token prediction
```

Causal masking prevents a token from attending to future tokens. The same autoregressive objective is used for pre-training and generation.

3. Evolution

GPT-1 (2018): introduced generative pre-training followed by supervised fine-tuning using an early decoder-only Transformer.
GPT-2 (2019): scaled model and data, demonstrating stronger zero-shot text generation.
GPT-3 (2020): much larger scale; the widely reported largest version had 175 billion parameters and demonstrated few-shot/in-context learning.
GPT-3.5 era: improved instruction following and conversational behaviour through instruction tuning and preference alignment.
GPT-4 (2023): improved capability and reasoning; supported image input in supported configurations alongside text.
GPT-4+ generation: continued improvements in multimodal interaction, tool use, context handling and efficiency across variants. Exact specifications vary and some are not publicly disclosed.

4. Scaling parameters and data

Increasing model capacity, training data and compute can improve the ability to represent language patterns when balanced appropriately. Scaling alone does not guarantee factual accuracy or safety; data quality, training and evaluation also matter.

5. Context length

The context window is the amount of tokenized information the model can consider in one interaction. Larger windows help with long documents and conversations, but increase compute requirements and do not guarantee perfect recall of every detail. Exact limits differ by model/version.

6. Multimodality

Early GPT models focused mainly on text. Later multimodal systems can process combinations of text and images; some model families support audio or other modalities. Vision-language systems map visual features into representations that can interact with language tokens, enabling image question answering and captioning.

7. Alignment and tools

Instruction tuning improves following user requests. Preference-based training improves helpfulness and response style. Tool-use capabilities allow supported models to call external tools or retrieve information. These features vary by model and configuration.

8. Conclusion

GPT evolved from GPT-1 to later generations through scaling, improved training and alignment, larger context support and multimodal capabilities, while retaining the decoder-only autoregressive foundation.

Q7. Describe the role of Transformers in Natural Language Processing (NLP) and explain at least two concrete real-world applications (Machine Translation, Abstractive Summarization).

1. Introduction and role

NLP enables computers to process and generate human language. Transformers use self-attention so each token can use information from other relevant tokens, including distant words. They provide contextual representations, model long-range relationships, allow parallel training across positions and support transfer learning through pre-trained models.

2. Transformer NLP pipeline

```
Text input → Tokenization → Embeddings + positions
      → Transformer attention blocks
      → Context-aware representations
      → Task-specific output / decoder
      → Translation, summary, answer or label
```

3. Application 1: Machine Translation

Machine translation converts text from one language to another while preserving meaning. In an encoder-decoder Transformer, the encoder reads the source sentence and produces contextual representations. The decoder generates the target sentence token by token while attending to encoder outputs.
Example: “Where is the railway station?” is converted into an equivalent sentence in the target language.
Steps: tokenize source; encode tokens; decoder attends to source representations; predict next target token; repeat until completion.
Benefits: captures context and word order and supports parallel training. **Challenges:** idioms, ambiguity, low-resource languages and domain terminology.

4. Application 2: Abstractive Summarization

Abstractive summarization creates a shorter version of a document using newly composed sentences. This differs from extractive summarization, which selects existing sentences. The encoder reads the document; the decoder attends to its representations and generates a concise summary token by token.
Example: a long flood report is summarized into a few sentences describing location, impact and rescue response.
Benefits: saves reading time and produces concise text. **Challenges:** may omit details or generate unsupported statements, so important summaries must be checked.

5. Other applications

Transformers are also used in sentiment analysis, named-entity recognition, chatbots, question answering, speech-text systems, information extraction, grammar correction and semantic search.

6. Advantages, limitations and conclusion

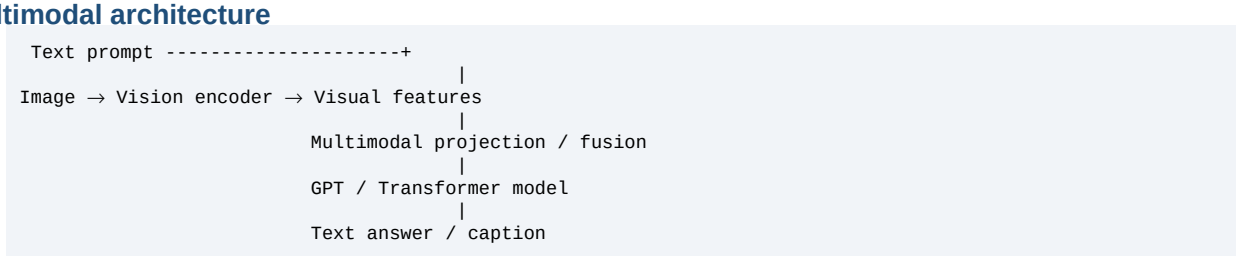
Advantages include contextual understanding, long-range dependency modelling, parallelizable training and transfer learning. Limitations include high attention cost for long sequences, bias and hallucination. Transformers are central to modern NLP and power applications such as translation and abstractive summarization.

Q8. Explain how GPT models are applied across multimodal domains: (a) Natural Language Processing, and (b) Computer Vision (Vision-Language Models / GPT-4V).

1. Introduction

GPT models use autoregressive Transformer language modelling. Multimodal systems extend this capability to process more than one data type. A Vision-Language Model (VLM) combines image information with language so a user can ask questions about an image and receive a text response.

2. Multimodal architecture



Exact internal designs vary. A common approach uses a vision encoder and a projection layer to map visual features into a representation the language model can use.

3. (a) NLP applications

GPT processes tokenized text and generates responses using next-token prediction. Applications include text generation (emails, reports and explanations), question answering, summarization, translation, coding assistance and conversational systems.

4. (b) Computer Vision / GPT-4V

Vision-language systems accept image inputs and combine visual representations with text instructions. GPT-4V is an example of GPT image-understanding capability in supported configurations. Applications include image question answering, image captioning, understanding charts and diagrams, interpreting screenshots/documents and visual educational assistance.

5. Example

A user uploads a bar chart and asks which month has the highest sales. The vision component extracts chart structure and values; the language model combines those visual features with the question and produces a natural-language answer. If the chart is unclear, the system should communicate uncertainty rather than invent a value.

6. Benefits and limitations

Benefits include natural interaction with images and text, document/chart analysis, accessibility and visual question answering. Limitations include misreading small text or complex charts, unsupported descriptions and privacy concerns. High-stakes medical, legal or safety decisions require expert verification.

7. Conclusion

GPT supports NLP through language modelling and can be extended into vision-language systems that interpret images with text. This enables generation, question answering, captioning and visual document understanding, with verification still essential.

Q9. What is Agentic AI? How does it differ from traditional AI and standard prompt-response generative models? Discuss at least three enterprise applications of Agentic AI.

1. Definition

Agentic AI describes AI systems that pursue goals through multi-step reasoning, planning, tool use, observation and feedback. Instead of only generating a response, an agent can execute a sequence of permitted actions, check results and adapt its plan.

2. Architecture

```
 Business / user goal
        |
Goal interpretation
        |
Planning and task breakdown
        |
Reasoning / decision
        |
Tools: APIs, apps, databases
        |
Observe results
        |
Evaluate progress
     /       \
Incomplete  Complete
    |           |
Re-plan     Report result
    +-- feedback loop
```

3. Difference from other AI

Traditional AI is often designed for a specific prediction, classification or detection task. Standard prompt-response generative AI mainly produces content in response to a prompt. Agentic AI combines a model with planning, state, tools and feedback to complete a larger workflow.

4. Comparison table

TABLE: Traditional AI | Predict/classify; usually task-specific; often fixed pipeline. Prompt-response GenAI | Generates text/content; commonly prompt → response. Agentic AI | Goal-directed multi-step workflow; selects tools; observes outcomes and replans.

Agentic systems can use APIs and enterprise software, but only within their granted permissions.

5. Core capabilities

Goal decomposition converts an objective into smaller tasks. Planning orders actions and handles dependencies. Tool use connects to APIs, browsers, databases and apps. Memory/state retains relevant task information where supported. Observation checks results. Feedback causes retries or replanning.

6. Enterprise application 1: Customer support

An agent can classify a request, retrieve order information from an authorized system, check delivery status, draft a response and route complex cases to a human. Refunds or account changes require policy controls and approval.

7. Enterprise application 2: IT operations

An IT agent can inspect alerts, gather logs, correlate events, consult runbooks, suggest remediation and verify recovery. It should escalate uncertain incidents and require approval for risky production changes.

8. Enterprise application 3: Finance / invoice processing

An agent can extract invoice details, compare them with purchase orders, identify missing fields, flag mismatches and prepare accounting records. Payment release should require authorized review and an audit trail.

9. Additional applications and governance

Other uses include sales lead research, CRM updates, HR onboarding, supply-chain monitoring, procurement comparisons and internal knowledge search. Benefits include faster workflows and reduced repetitive work. Risks include incorrect actions, prompt injection, data leakage and excessive permissions. Controls include least-privilege access, tool allowlists, validation, logging, monitoring and human approval for high-impact actions.

10. Conclusion

Agentic AI extends prediction and content generation into goal-directed workflows. Customer support, IT operations and finance are important enterprise applications, provided governance and human oversight are maintained.